

Part 12: Morris Traversal

Inorder in $O(1)$ extra space

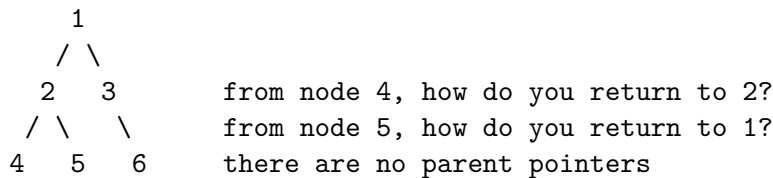
Read this part only after iterative inorder (Part 4) feels comfortable. Morris traversal is the one advanced item in this course. You do not need it for most problems, but it is a favourite interview question and it teaches something real about what a stack is for.

The question it answers

Both inorder implementations we know use $O(H)$ extra memory:

- recursion uses the call stack
- the iterative version uses an explicit `stack<TreeNode*>`

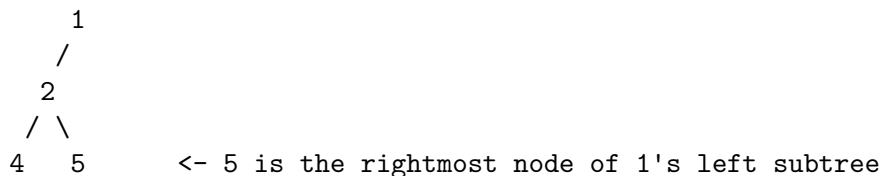
Why is that memory needed at all? Because of one specific moment. You are at node 1, you dive into the left subtree, and when the left subtree finishes you have to **get back to node 1**. Nothing inside the left subtree points upward. Without a stack, once you go down you cannot come back.



Morris traversal's answer: **build the return path into the tree itself, temporarily, and remove it afterwards.**

The key observation

Take node 1. The last node visited in its left subtree, in inorder, is node 5. In general, the last node of a left subtree in inorder is its **rightmost** node, reached by going left once and then right as far as possible. That node is called the **inorder predecessor** of node 1.



Here is the point: node 5 has an empty right pointer. That pointer is currently wasted. So before diving into the left subtree, set

```
5->right = 1
```

Now, when the traversal walks off the end of the left subtree, it lands on node 1 automatically. No stack needed. A pointer used this way is called a **thread**, which is why this is sometimes called threaded tree traversal.

Of course, we must remove the thread later, or we have created a cycle and left the tree corrupted.

The algorithm

```
curr = root
while curr is not NULL:

    if curr has no left child:
        visit curr
        curr = curr.right           // may be a real edge or a thread

    else:
        pred = rightmost node of curr's left subtree
            (go left once, then right while right exists and right != curr)

        if pred.right == NULL:      // first time here
            pred.right = curr        // create the thread
            curr = curr.left         // go explore the left subtree

        else:                       // pred.right == curr, second time here
            pred.right = NULL        // remove the thread, restore the tree
            visit curr               // left subtree is finished, so now visit
            curr = curr.right        // move on to the right subtree
```

The `pred.right != curr` condition inside the search is essential. Without it, once a thread exists the search loops forever along it.

The whole design turns on one fact: **you arrive at every node with a left child exactly twice**. The first arrival means “left subtree not started”, so you build the thread and go left. The second arrival means “left subtree finished”, so you tear the thread down and finally visit the node. The thread’s presence or absence is the memory. That is what replaces the stack.

C++

```
vector<int> morrisInorder(TreeNode* root) {
    vector<int> out;
    TreeNode* curr = root;

    while (curr != NULL) {
```

```

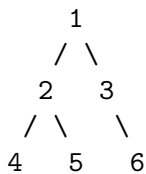
    if (curr->left == NULL) {
        out.push_back(curr->val);
        curr = curr->right;
    } else {
        TreeNode* pred = curr->left;
        while (pred->right != NULL && pred->right != curr)
            pred = pred->right;

        if (pred->right == NULL) {           // first visit: thread it and go left
            pred->right = curr;
            curr = curr->left;
        } else {                             // second visit: unthread, visit, go right
            pred->right = NULL;
            out.push_back(curr->val);
            curr = curr->right;
        }
    }
}
return out;
}

```

Twelve lines. No stack, no recursion.

Dry run



curr=1 has left	pred = rightmost of {2,4,5} = 5, 5.right is NULL thread 5.right = 1, curr = 2
curr=2 has left	pred = rightmost of {4} = 4, 4.right is NULL thread 4.right = 2, curr = 4
curr=4 no left	visit 4 out = [4] curr = 4.right = 2 (thread)
curr=2 has left	pred search: 4.right == 2 == curr, stop unthread 4.right = NULL, visit 2 out = [4,2] curr = 2.right = 5
curr=5 no left	visit 5 out = [4,2,5] curr = 5.right = 1 (thread)
curr=1 has left	pred search: 2 -> 5, 5.right == 1 == curr, stop unthread 5.right = NULL, visit 1 out = [4,2,5,1] curr = 1.right = 3

curr=3	no left	visit 3	out = [4,2,5,1,3]
		curr = 3.right = 6	
curr=6	no left	visit 6	out = [4,2,5,1,3,6]
		curr = NULL, loop ends	

Output 4 2 5 1 3 6, the correct inorder, and the tree is exactly as it was at the start. Every thread that was created was removed.

Complexity

Space: $O(1)$. Two pointers, curr and pred. Nothing grows with N. The output vector does not count, since it is the answer.

Time: $O(N)$. This surprises people because of the inner while loop that walks down to find the predecessor. The careful argument: every edge of the tree is traversed at most three times, once when searching for a predecessor the first time, once when searching the second time, and once when moving normally. A tree has $N - 1$ edges, so the total work is at most about $3N$ steps. Constant factor larger than plain recursion, same asymptotic class.

Morris preorder

Preorder needs the node recorded on the **first** arrival instead of the second. So move the push_back into the branch that creates the thread:

```
vector<int> morrisPreorder(TreeNode* root) {
    vector<int> out;
    TreeNode* curr = root;

    while (curr != NULL) {
        if (curr->left == NULL) {
            out.push_back(curr->val);
            curr = curr->right;
        } else {
            TreeNode* pred = curr->left;
            while (pred->right != NULL && pred->right != curr)
                pred = pred->right;

            if (pred->right == NULL) {
                out.push_back(curr->val);    // record on the FIRST visit
                pred->right = curr;
                curr = curr->left;
            } else {
                pred->right = NULL;
                curr = curr->right;
            }
        }
    }
}
```

```
    return out;  
}
```

On our tree this gives 1 2 4 5 3 6. Compare it with the inorder version: exactly one line moved. This is the same “first touch versus second touch” idea from the walk-around-the-tree picture in Part 2, now implemented literally.

Morris postorder also exists. It requires reversing the right-spine of a subtree, printing it, and reversing it back. It is fiddly, rarely asked, and not worth memorising at this stage. Know that it exists.

Risks and honest drawbacks

- **It modifies the tree while running.** If your code throws an exception, returns early, or is interrupted in the middle, the tree is left with dangling threads, which are cycles. Never insert a return in the middle of a Morris loop.
- **It is not safe with concurrent readers.** Another thread reading the tree during traversal sees a corrupted structure.
- **It cannot run on a const tree.** You need write access even though you are only reading.
- **It is harder to debug.** A single wrong condition creates an infinite loop rather than a wrong answer.
- **The constant factor is worse** than plain recursion because of the predecessor searches.

So when is it actually worth it? When the tree is huge and deep enough that $O(H)$ stack space genuinely matters, or when the interviewer asks for $O(1)$ space inorder. In ordinary contest and interview coding, use recursion.

What you are expected to know

You do not need to reproduce Morris traversal from memory under time pressure. You should be able to:

1. Explain **why** normal traversal needs a stack, in one sentence: there is no way back up.
2. Explain the fix: thread the inorder predecessor’s empty right pointer to the current node.
3. Explain how the code knows whether it is arriving for the first or second time.
4. State the complexities and why time is still $O(N)$.
5. Write the standard implementation with a little thought, having understood it.

Common mistakes

- **Omitting `pred->right != curr` in the predecessor search.** Infinite loop along an existing thread.
- **Forgetting to remove the thread.** You return the right answer and leave a cycle in the tree, which breaks everything afterwards.
- **Recording the node in the wrong branch.** First branch gives preorder, second gives inorder. Choose deliberately.

- **Moving curr to curr->left in the unthreading branch.** After unthreading, the left subtree is finished. Go right.
- **Claiming the time is $O(N \log N)$** because of the inner loop. Use the edge counting argument.
- **Using Morris just to look clever** on a problem where recursion is clearer.

Check yourself

1. What exact problem does the thread solve?
2. Which node do we thread from, and why is its right pointer guaranteed to be free?
3. How does the algorithm distinguish a first arrival from a second arrival at the same node?
4. Why is the time complexity $O(N)$ despite the inner search loop?
5. Which single line moves to convert Morris inorder into Morris preorder?
6. Name two situations where Morris traversal would be a bad choice.